and division, respectively. If you want further clarity, use the function *pp( )* to find the values of *b*, *c*, and *d*. Line 12 is very critical. It defines a function named *f( )* using the function *function( )* of Theano. The function *function( )* takes two parameters. Parameter 1 is the data to be operated upon and parameter 2 is the function to be applied on the data given as parameter 1. Here, the data is the two symbolic scalar variables *x* and *y*. The function is given by *[a b c d]* which represents symbolic addition, subtraction, multiplication, and division, respectively. Finally, in Line 13, the function *f( )* is provided with actual values rather than symbolic ones. The output of this operation is also shown in Figure 3. It is very easy to verify that the output shown is correct.

Now, let us see how matrices can be created and manipulated using Theano. Consider the code shown in Figure 4. It is important to note that the first three lines of code shown in Figure 3 should be added here also, if you are writing this program independent of the programs we have discussed so far. Now, let us try to understand the code. Line 1 of the code creates two symbolic matrices *x* and *y*. Think of *x* and *y* as two 0-dimensional arrays. But this time these matrices are of type integer unlike the last time where the data type was double. Further, this time a plural constructor (*imatrices*) is used so that more than one matrix can be constructed at the same time. Lines 3 to 5 perform symbolic addition, subtraction, and multiplication, respectively, on symbolic matrices *x* and *y*. Here again you can use *print(pp(a))*, *print(pp(b))*, and *print(pp(c))* to better understand the symbolic nature of the operations being performed. If you add these lines of code, you will get *(x+y)*, *(x-y)*, and *(x \dot y)*, respectively, as output. Line 5 generates a function *f( )* which takes two parameters as before. Again, parameter 1 is the data to be operated upon and parameter 2

```
1  import theano
2  import theano.tensor as T
3  from theano import function
4  from theano import pp
5  x = T.dscalar('x')
6  y = T.dscalar('y')
7  a = x + y
8  print(pp(a))
9  b = x - y
10 c = x * y
11 d = x / y
12 f = function([x, y], [a, b, c, d])
13 f(333, 222)
```

```
(x + y)
[array(555.), array(111.), array(73926.), array(1.5)]
```

Figure 3: Our first code using Theano

```
1 x, y = T.imatrices('x', 'y')
2 a = x + y
3 b = x - y
4 c = x.dot(y)
5 f = function([x, y], [a, b, c])
6 f([[1, 2],[3, 4]], [[5, 6],[7, 8]])
```

```
[array([[ 6,  8],
        [10, 12]], dtype=int32),
 array([[-4, -4],
        [-4, -4]], dtype=int32),
 array([[19, 22],
        [43, 50]], dtype=int32)]
```

Figure 4: Manipulating matrices with Theano

```
1 import numpy as np
2 print(np.mean(C1), np.std(C1))
3 print(np.mean(C2), np.std(C2))
```

```
21.0 1.0
21.666666666666668 11.813363431112899
```

Figure 5: Arithmetic mean and standard deviation

is the function to be applied on the data given as parameter 1. But this time, the data is the two symbolic matrices *x* and *y*. The function is given by *[a b c]* which represents symbolic addition, subtraction, and multiplication, respectively. Finally, in Line 6, the function *f( )* is provided with actual values rather than symbolic ones. The two matrices given as input to *f( )* are [[1, 2], [3, 4]] and [[5, 6], [7, 8]]. The output of this operation is also shown in Figure 4. It is very easy to verify that the three output matrices shown are correct. Notice that, in addition to scalars and matrices (for both of which we saw examples now), *tensor* also offers constructors like vector, row, column, different types of tensors, etc. Let us stop discussing Theano for now and revisit it while discussing advanced topics in probability and statistics.

Baby steps with probability

Now, let us continue discussing probability and statistics. I had suggested in the last article (while introducing probability) that you carefully go through three Wikipedia articles, and with that assumption of familiarity I tried to motivate you by discussing the Normal distribution. However, we must revisit some of the basic notions of probability and statistics before we start developing AI and machine learning based applications. I hope all of you have heard about arithmetic mean and standard deviation (SD). Arithmetic mean can be thought of as the average of a set of values. SD can be thought of as the variation or dispersion of a set of values. If the SD value is low then the elements in the set tend to be closer to the mean. On the contrary, if the SD value is high then the elements of the set are spread out over a wider range. But how can we calculate arithmetic mean and SD using Python? There is a module called *statistics* available with Python which can be used to find mean and standard deviation. However, experts are of the opinion that this module is very slow and hence we choose *NumPy*. Now, consider the code shown in Figure 5.

The code prints the mean and standard deviation of two lists C1 and C2 (whose values are hidden from you for the time being because of obvious reasons). What interpretations can you make from these values? Nothing! They are just numbers for you right now. But what if I tell you the lists contain the marks of six students, studying in 10th standard in school A and school B, for an exam in mathematics (the exam is out of 50 with a pass mark of 20). The mean value tells us that students from both the schools have relatively poor average marks with